ECE 342 Project Overview

Introduction

In ECE 341, you spent six weeks working with existing block designs on your own. You then spent four weeks designing a small system with a team from scratch. **This term, you will spend all ten weeks working toward delivering one system.** This will include, but is not limited to: identifying the engineering requirements, designing system-level, top-level, and block-level block diagrams, creating well-defined interfaces to satisfy the engineering requirements, designing blocks to realize the interfaces, validating the interfaces, building the blocks, keeping detailed documentation of the entire process, and finally verifying each block before integrating and verifying the system as a whole.

The course textbooks are:

- The OSU ECE Design Coursebook Pts. 1-3
  - Pt. 1: Project Definition and Requirements
  - Pt. 2: System Design and Project Artifacts
  - Pt. 3: Testing and System Integration
- Design for Electrical and Computer Engineering

Your team will meet weekly in lecture to work through structured worksheets which will help to guide you through the design process. Labs will be structured or unstructured depending on the week (see Canvas).

The general course structure is as follows:

TABLE I
ECE 342 COURSE STRUCTURE

Week		
1	Getting Started	Project/Team Selection
2		Project Proposal
3	Block 1 Design Cycle	Design, Validation, and Documentation
4		Build and Peer Review
5		Design Verification
6	Block 2 Design cycle	Design, Validation, and Documentation
7		Build and Peer Review
8		Design Verification
9	System Delivery	System Integration
10		System Verification
11	No Final!	

Forming a Project Team

To get started, select your team of three. Each team member will be responsible for at least two blocks in the system. You are free to work with students outside of your usual lab section, but make sure your team has availability to meet as a group often enough to get the work done. **You will fill out a form during lecture in week 1 selecting your preferred projects and team members.** Team and project assignments will be made prioritizing your selections. However, there is no guarantee that you will be placed on the team or project you select. If you have no preference, you will be assigned to a team and project at random.

Project Management

The three main objectives in any engineering design project are to deliver a project that meets the requirements of the customer/user and is delivered on-time and under-budget [1]. Delivering a project that meets these objectives will require a significant amount of coordination. The following steps may help you start on the right track:

1. **Determine a method of communication** for your team that works for all members, such as email, Teams, Slack, Discord, a group chat, etc.
2. **Make a timeline** of the project, noting any deadlines in the course and when any additional parts should be ordered by.
3. **Arrange a regular meeting time** to check in with your team members and evaluate the state of the project.
4. **Set a work schedule** and identify tasks for each team member.
5. **Employ tools covered in ECE 341** like work breakdown structure, network diagrams, and Gantt charts.
6. **Document these processes** so you can fill out the Team Member Work Distribution table in the System Document.

Selecting a Project

You may choose from any of the following projects. Be sure you understand the engineering requirements and associated constraints.

Custom Detection Timer

Description and Constraints

For this project, the team will be required to make a countdown timer. This project requires a custom-made enclosure (3D printed or laser cut is recommended) with associated design files. This timer must include at least 5 minute and 10 minute settings with remaining time displayed. The timer must detect an arbitrary object (such as a phone) placed on or near the timer, and must sound an alarm and stop the countdown when the detected object is removed.

Requirements

1. **Customer Requirement: The system must be accurate.**
   Engineering Requirement: The system output must be accurate to a precision of +/- 1 second when responding to the removal of the detected object.

2. **Customer Requirement: The system must be safe.**
   Engineering Requirement: The system must use connectors for every module used in the timer system, have a power disconnect switch, and not have any exposed conductors. All devices must be rated at least IP40 (https://en.wikipedia.org/wiki/IP_Code).

3. **Customer Requirement: The system must be intuitive.**
   Engineering Requirement: At least 9/10 users will use the timer without needing instructions.

4. **Customer Requirement: The alarm beep must have an easy tone to hear.**
   Engineering Requirement: The system must be 440 Hz +/- 1 Hz.

5. **Customer Requirement: The system must have dimmable timer lights.**
   Engineering Requirement: The system must include 3 brightness levels that are user-selectable.

Oscilloscope

Description and Constraints

For this project, the group will build an oscilloscope. This project requires a custom-made enclosure (3D printed or laser cut is recommended) with associated design files. It must sample at a rate of 200 kHz with multiple channels. The time and voltage axes must be adjustable and pausable. The oscilloscope must include a trigger function.

Requirements

1. **Customer Requirement: The oscilloscope must have multiple channels.**
   Engineering Requirement: The system will have at least two channels that can function simultaneously and independently.

2. **Customer Requirement: The system must be modular.**
   Engineering Requirement: The system must connect and disconnect from the oscilloscope probes using robust connectors.

3. **Customer Requirement: The system must have a capable user interface.**
   Engineering Requirement: The system must include a configurable trigger, adjustable time, and adjustable voltage axis.

4. **Customer Requirement: The oscilloscope must be responsive.**
   Engineering Requirement: The system must respond to user input in under 100 milliseconds.

5. **Customer Requirement: The oscilloscope must have good fidelity.**
   Engineering Requirement: The system must sample at a rate of at least 200 kHz independently on all channels.

Two-Axis Robotic Arm

Description and Constraints

For this project, the group will be asked to develop a robotic arm that can draw pictures on a 8.5” x 11” sheet of paper. This project involves embedded design, control systems, and MATLAB or Python scripting.

Requirements

1. **Customer Requirement: The system should be fast.**
   Engineering Requirement: The system must draw with a speed of 4 inches per second.

2. **Customer Requirement: The system must be accurate.**
   Engineering Requirement: The system must draw a 10 inch straight line +/- .25 inch in all directions.

3. **Customer Requirement: The system needs to be inexpensive and manageable to manufacture.**
   Engineering Requirement: The system will use a SCARA topology, with two rotating joints to control arm actuation.

4. **Customer Requirement: The system must have a commonly known interface.**
   Engineering Requirement: The system input will be G-code commands. These commands must be made available within the Python or MATLAB GUI.
   G0, G1, G90, G91, G20, G21, M2, M6.

5. **Customer Requirement: The system must have an emergency stop.**
   Engineering Requirement: The system will stop moving in less than 1 second after user input and then return to normal operation when the emergency stop input is reset.

Universal System Constraints

The following constraints apply to all projects:

1. **The system may not include a breadboard.**
   Anytime you show any part of your system for a score, you *may not* use a breadboard. Feel free to use them for testing and development, just not for check-offs.
2. **The system must include a custom student-designed PCB.**
   The PCB must be functionally important to the system. The PCB must be designed for this system. The PCB does not need to be perfect and might have modifications to it. This is expected.
3. **All connections to PCBs must use connectors.**
   No wires will be directly soldered to any PCB that you design or purchase.
4. **Any power input to the system must be DC.**
   AC power may not be used to power the system.
5. **All power supplies must be at least 65% efficient.**
   Modern electronics design must always be aware of power consumption and waste. This applies only to power supplies in the system, not the system overall.

Getting Started

Identifying Engineering Requirements

The first step in any engineering design project is identifying the engineering requirements. Typically, these are synthesized from the given customer requirements by the engineering team. **For your project, you are given five customer requirements and their resulting engineering requirements. You will then create two additional customer and engineering requirements.** Go back and read through the description, constraints, and requirements for your project. Identify the engineering requirements and begin to consider what additional customer and engineering requirements may be created. Keep the universal system constraints in mind.

Creating Additional Engineering Requirements

IEEE Standards state that all engineering requirements should meet the following four properties [1]:

1. **Abstract:** The requirement should state what the system should do, not how it should do it. Avoid mentioning specific components or implementations whenever possible. In other words, requirements should be *implementation independent* [2].
2. **Verifiable:** The requirement should be testable. There must be a way to show the requirement is met by measurement or demonstration.
3. **Unambiguous:** The requirement should leave no room for interpretation [2]. The engineer and customer/stakeholder should ideally never be able to disagree on whether a requirement is satisfied or not.
4. **Traceable:** There needs to be a clear reason why the engineering requirement exists. There should be a way to determine what customer requirement is the source of the engineering requirement [2].

For your additional engineering requirements, brainstorm some customer requirements that you might have if you were the customer. Then, synthesize those customer requirements into engineering requirements that meet the four properties listed above. You will submit two additional engineering requirements for approval in your project proposal.

Creating Block Diagrams and Defining Interfaces

Block diagrams should be generated on a white background and have all interfaces labeled. Interfaces should follow the **from_to_type** naming convention, use the types defined in the coursebook, and have *relevant* properties. For example, a DC power interface from a supply to a microcontroller should be named something like **supply_microcontroller_dcpwr** and have properties Vmin, Vmax, Inom, and Ipeak. It would be inappropriate to give this interface a property like Logic Level or Datarate, since those properties would be irrelevant to this type of interface. Think critically about defining interfaces. The more care and attention you put into this in the initial design phase the better. Note that you *may* use properties not listed in the coursebook as long as they are relevant to the interface. **You *may not* use interface types not listed in the coursebook.**

System-Level Block Diagram

System-level interfaces go from your system to outside your system or vice versa. They are defined to provide a way to satisfy the engineering requirements. As such, the system-level block diagram is often the next task once the engineering requirements have been identified/created.

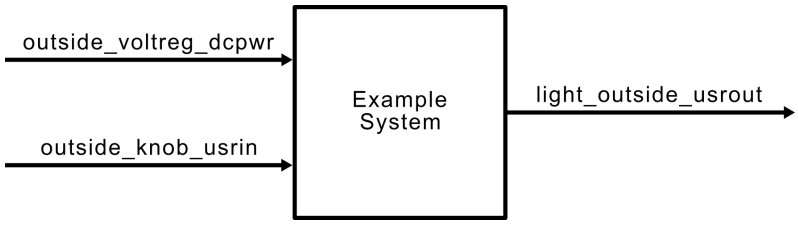

Fig. 1 Example of a system-level block diagram.

The system-level block diagram should be a "black-box" diagram of the system. It should have the entire system encapsulated in one block with only the system-level interfaces shown. Ideally, each interface should correspond to at least one engineering requirement. If an interface does not correspond to any engineering requirement, it should be functionally important to the system.

Top-Level Architecture Block Diagram

Once the system-level interfaces are defined and the system-level block diagram is generated, it's time to start thinking about the internal blocks of the system. **The top-level architecture block diagram should be a diagram showing each internal block in the system with each of its corresponding interfaces labeled.** It should include all system-level and internal interfaces.

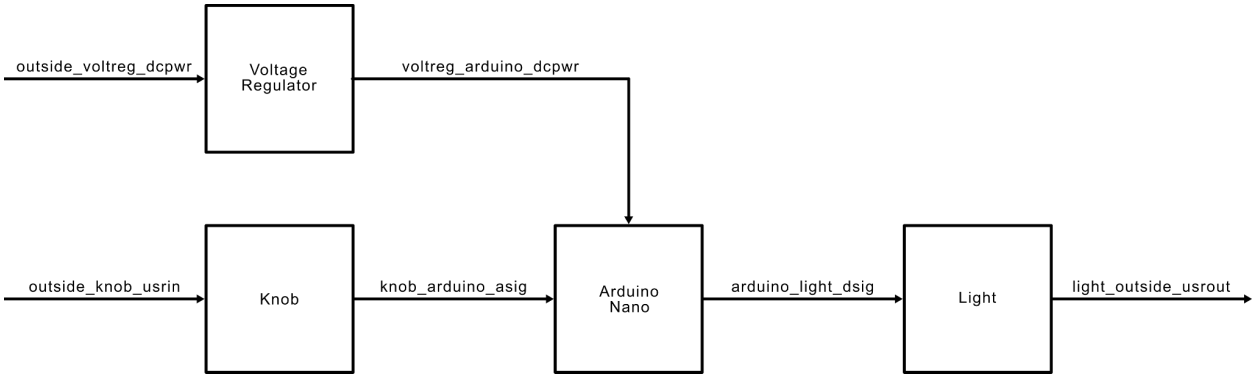

Fig. 2 Example of a top-level architecture block diagram

While developing the top-level architecture of your system. **Keep in mind that each block should be a similar amount of work and functionally important to the system.** The top-level architecture provides a way to both brainstorm the functionality of the project and split up the work evenly. Thinking about blocks as units of work can turn your diagram into a useful project management tool.

Once you have developed a feasible top-level architecture, you should start considering what blocks each team member will be responsible for. Remember, just because a team member is particularly familiar with one type of block doesn't necessarily mean they should work on that block by default. A successful team will make sure each team member is supported through the design process. Take the opportunity to learn from each other and expand your repertoires.

Additionally, keep in mind that your top-level architecture will likely need to be updated at some point in the term as you navigate unforeseen challenges. This is expected. Be sure to communicate with your team and ensure that interfaces/properties are updated as needed.

Block-Level Block Diagram

Each individual block in the top-level architecture should have its own block-level block diagram. This is a single block with each of its corresponding interfaces.

Creating a Project Proposal

Once you have identified the engineering requirements, brainstormed some interfaces to satisfy the requirements, and developed your first revisions of the system-level and top-level architecture block diagrams, you will submit a project proposal. A good project proposal should give an overview of the project and address the design goals and objectives of the engineering team. It should address the engineering requirements and justify their selection. Lastly, it should present a preliminary design to help demonstrate feasibility of the proposed system.

Documentation

In ECE 342 all documents must use IEEE figure and table labels, equation numbers, in-text citations, and references.

Block Design Documents

You will individually generate block-design documents for *each block* in your system. It is highly recommended that you use the provided template. Each block-design document must include at minimum:

- Video Verification Link
    - Create a video of your block verification process. This can be used if something catastrophic happens to your block before your check-off appointment. It is not an alternative to a live check-off.
- Description
    - 1-2 paragraphs describing *what the block does*.
    - Include a block-level block diagram here.
- Design Details
    - 1 or more pages describing *how the block works*.
    - Include a schematic or some other representative figure here.
    - Describe each interface and its function.
- Interface Validation
    - Include a table for each interface going to and/or from the block.
    - Each interface should have:
        - At least three *relevant* properties.
        - A description of why each property has its defined value.
        - A description of how the design will satisfy each property.
- Verification Process
    - Enumerate a verification process that an engineer of a similar experience level could execute without additional guidance.
    - Include Setup and Test sections.
    - Explicitly mention when an *interface or property* is verified.
- Artifacts
    - Provide a collection of resources you used in your design or previous revisions. Anything you found useful, but wasn't important enough for the design details section can go here.
- Future Recommendations
    - Reflect on your work. Give advice to future students (or your past self).
- References
    - List all references used in the design process in IEEE format.

System Document

As a team, you will generate a system document. It is highly recommended that you use the provided template. This will include at minimum:

- Video Verification Link
    - Create a video of your system verification process. This can be used if something catastrophic happens to your block before your check-off appointment. It is not an alternative to a live check-off.
- Team Member Work Distribution
    - Include a table describing what each member did and how many hours they worked on the project.
- Engineering Requirements

        - Enumerate the engineering requirements including the additional proposed requirements.
- System-level block diagram.
    - Include a system-level block diagram showing all system-level interfaces.
- System Description
    - 1-2 paragraphs describing *what the system does*.
    - Include a system-level block diagram here.
- System Design Details and Validation
    - Include your top-level architecture diagram here.
    - System Design Synthesis
        - 1 or more pages describing *how the system works.*
        - Describe each system-level interface and its function.
    - Block Design Details List
        - Provide links to each of your block design documents.
- System Level Interface Validation Table
    - Include a table for each interface going to and/or from the system.
    - Each interface should have:
        - At least three *relevant* properties.
        - A description of why each property has its defined value.
        - A description of how the design will satisfy each property.
    - These should match the tables in their respective block design documents.
- Verification Process
    - Enumerate a verification process that an engineer of a similar experience level could execute without additional guidance.
    - Include Setup and Test sections.
    - Explicitly mention when an *engineering requirement* is verified.
- Future Recommendations
    - Reflect on your work. Give advice to future students (or your past self).
- References
    - List all references used in the design process in IEEE format.

Block Design Cycle

Each block design cycle will take three weeks to complete. In each design cycle you will *individually* design, validate, document, build, peer review and verify one block.

Design, Validation, and Documentation

During the first week of each block design cycle you will design and validate one block to satisfy the interfaces required by the top-level architecture. Remember, **validation is showing *how you know the design will work.*** You should use datasheets, simulations, and calculations to show your design will satisfy the interface properties. At the end of the first week you will submit an initial block design document. Keep your document updated as you make changes to the design.

Build and Peer Review

The next week you will assemble your block. At the same time you will peer review classmate's blocks. Peer reviews should be kind, effective and constructive.

Verification

In the final week of each design cycle you will verify your block. Remember, **verification is showing *how the design did work*.** You need to show unambiguously how each interface property has been satisfied by the design. **This means taking measurements and thoroughly testing edge cases** while the block is isolated from the rest of the system. You may submit a final version of your block design document up to 10 minutes prior to your scheduled check-off appointment. Your verification will be graded based on the updated document. Note that the video verification link in your document can be used in the event that something catastrophic happens to your system before your check-off. It is not an alternative to a live check-off.

System Delivery

The final two weeks of term are dedicated to delivering your system. First you will perform system integration by assembling all of your blocks. The following week you will verify the system.

System Integration

You will assemble your system by connecting each of your blocks interfaces. If properties were well-defined and each block was properly verified, system integration should be straight-forward. However, this is often not the case. Unexpected issues often occur that need to be addressed. This is why an entire week of the term is dedicated to this phase.

System Verification

System verification is performed in week ten. Like block verifications, this is a live check-off. However, instead of verifying interfaces, you will be verifying your engineering requirements. Remember, **verification is showing *how the design did work*.** You need to show unambiguously how each engineering requirement has been satisfied by the design. **This means taking measurements and thoroughly testing edge cases.** You may submit a final version of your system document up to 10 minutes prior to your scheduled check-off appointment. Your verification will be graded based on the updated document. Note that the video verification link in your document can be used in the event that something catastrophic happens to your system before your check-off. It is not an alternative to a live check-off.

Conclusion

At the end of the term, you will have designed, validated, built, and verified your system. Take a moment to reflect on all the hard work you and your team did to get here.

References

[1] “Design For Electrical and Computer Engineering,” OER Commons. Accessed: Mar. 17, 2026. [Online]. Available: https://oercommons.org/courses/coulston-design-for-electrical-and-computer-engineering-design-for-ece-text-and-instructor-resources

[2] "IEEE Guide for Developing System Requirements Specifications," in IEEE Std 1233-1996 , vol., no., pp.1-30, 22 Dec. 1996, doi: 10.1109/IEEESTD.1996.81000.
keywords: {Software requirements and specifications;IEEE standards;Software design/development;SyRS;requirement;system},